UFMG

Universidade Federal de Minas Gerais
Prof. Fátima Machado de Souza Lima

Estudo de Caso 01
Problema do Caixeiro Viajante (TSP)

Neste estudo de caso trataremos o problema do caixeiro viajante, do inglês Travel Salesmam Problem (TSP). Os primeiros estudos desse problema tiveram início no século XVIII por um matemático irlandês chamado Sir William Rowam Hamilton e um matemático britânico chamado Thomas Penyngton Kirkman. Uma discussão mais detalhada sobre o trabalho de Hamilton & Kirkman pode ser encontrado no livro intitulado Graphs Theory (Biggs et al. ((1986))).

Acredita-se que a forma geral do TSP foi estudada pela primeira vez por Kalr Menger em Viena e Harvard. O problema foi posteriormente desenvolvido por Hassler, Whitney & Merrill em Princeton. Uma descrição mais detalhada sobre a conexão entre Menger & Whitney e o desenvolvimento do TSP pode ser encontrada em Schrijver ((1960)) (Matai et al. ((2010))).

Sua origem está relacionada a um caixeiro viajante que pretende visitar um conjunto de cidades, somente uma vez cada, partindo de um determinado ponto e retornando a esse mesmo local empregando o menor tempo/custo possível.

O problema, que parece simples, é um dos mais estudados na área da matemática computacional em razão de sua vasta aplicação e complexidade de obtenção de uma solução ótima. Os códigos do problema estão disponíveis no link: `https://pifop.com/app/view/gbujJFcoq6ND0lRj8wjD`.

1 Formato de entrega

1. Para este estudo vocês devem apresentar 5 aplicações práticas do TSP, se possível citar artigos.
2. Interpretar o modelo que contém as restrições de subciclos.
3. Rodar 3 instâncias: a instância dada no próprio estudo de caso e gerar duas outras, uma com 30 pontos e outra com 50 pontos.
4. O relatório deve conter as intâncias geradas, os resultados e uma breve discussão sobre os resutados obtidos (observar o tempo de resolução de cada instância e reportar no relatório).
5. Deve ser entregue um relatório com a respostas até o dia 17 de abril de 2026, às 23:59, exclusivamente pelo Moodle.
6. O relatório com as respostas deve ser entregue no formato pdf, exclusivamente via Moodle, no link disponível.
7. O template (latex) para construção do relatório está disponível no Moodle.
8. No caso do aluno que optar por não fazer o relatório em latex, basta seguir o formato do arquivo em *template.pdf* que também se encontra disponível no Moodle.

2 O caso

Para estudarmos este problema, vamos considerar que um laticínio possui uma fábrica e 20 lojas de varejo, as quais visita semanalmente para repor estoque. O caminhão que faz as entregas não tem restrição de capacidade, parte da fábrica, visita todos os clientes e retorna ao ponto original de partida, a fábrica. Essa rota deve ser definida de modo que a distância percorrida seja a menor possível. As coordenadas geográficas da fábrica (planta), bem como das lojas de varejo visitadas são dadas na Tabela 1 e uma representação gráfica do problema é mostrada na Figura 1.

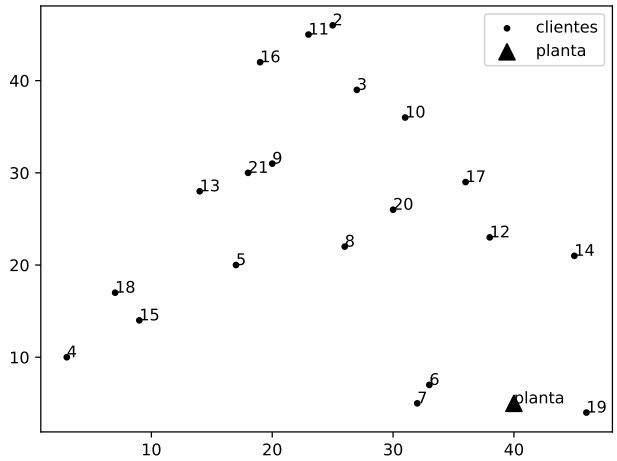

Figura 1: Representação gráfica do estudo de caso.

Tabela 1: Coordenadas dos clientes e fábrica.

Ponto	Coord x	Coord y	Ponto	Coord x	Coord y
0	40.0	5.0	10	23.0	45.0
1	25.0	46.0	11	38.0	23.0
2	27.0	39.0	12	14.0	28.0
3	3.0	10.0	13	45.0	21.0
4	17.0	20.0	14	9.0	14.0
5	33.0	7.0	15	19.0	42.0
6	32.0	5.0	16	36.0	29.0
7	26.0	22.0	17	7.0	17.0
8	20.0	31.0	18	46.0	4.0
9	31.0	36.0	19	30.0	26.0
			20	18.0	30.0

3 Notação, Definições e Modelo

O problema consiste em projetar uma rota de entrega de menor custo ou distância que sai do depósito para atender um conjunto de clientes espalhados numa área geográfica por um único veículo não capacitado e cada cliente deve ser visitado exatamente uma única vez.

Para construirmos o modelo vamos considerar um grafo direcionado $G = (N, A)$, exemplificado na Figura 2, no qual $N = 0, \ldots, n$ e $A = \{(i, j) \in N \times N : i \neq j\}$ representam os conjuntos de nós e arcos. O nó 0 representa o depósito, enquanto os demais nós correspondem aos nós dos clientes. A matriz de

custo c_{ij} é definida em A, não sendo necessariamente simétrica, ou seja, c_{ij} pode ser diferente de c_{ji}. Por simplificação, assumiremos que a matriz é simétrica e que o tempo de viagem ($l_{ij} \geq 0$) e a distância ($c_{ij} \geq 0$) entre os nós i e j são equivalentes ($l_{ij} = c_{ij}$).

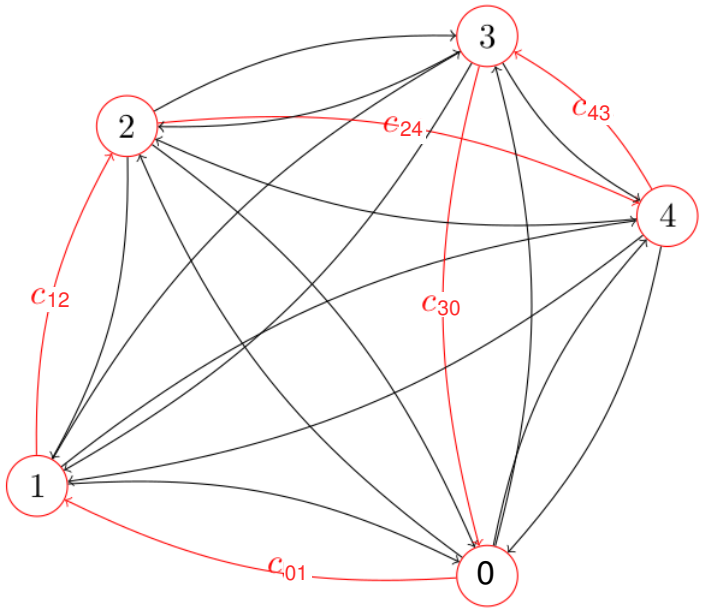

Figura 2: Exemplo de um grafo.

Para esse problema consideramos duas variáveis de decisão, $x_{ij} \in \{0, 1\}$ igual a 1 se o arco $(i, j) \in A$ é usado na rota ótima; 0, caso contrário e $f_{ij} \geq 0$ que é a quantidade de fluxo fictício passando pelo arco $(i, j) \in A$. A Tabela 2 mostra de maneira resumida as variáveis e os parâmetros do modelo.

Tabela 2: Lista de parâmetros e variáveis..

Conjuntos
$N : \{1, \ldots, n\}$: conjunto de pontos de destino.
$A : \{(i, j) : i, j \in N \wedge i \neq j\}$: conjunto de arcos.
Parâmetros
0 : depósito.
n : número de pontos de destino.
c_{ij} : custo de transportar uma unidade de produto de i para j. $\forall (i, j) \in A$.
Variáveis de Decisão
x_{ij} : binária. Se o arco $(i, j) \in A$ estiver presente na rota x_{ij} é igual a 1, caso contrário, igual a 0.
f_{ij} : ≥ 0. Quantidade de fuxo fictício que passa pelo arco $(i, j) \in A$.

Mas por que utlizar a variável de fluxo f_{ij} se não existe demanda e queremos apenas definir as rotas? Vejamos. O problema inicialmente pode ser modelado da seguinte forma:

$$\min \sum_{(i,j)\in A} c_{ij}x_{ij} \tag{1}$$

$$\text{sujeito a: } \sum_{(i,j)\in A} x_{ij} = 1 \qquad \forall i \in N \tag{2}$$

$$\sum_{(i,j)\in A} x_{ij} = 1 \qquad \forall j \in N \tag{3}$$

$$x_{ij} \in \{0,1\} \qquad \forall (i,j) \in A \tag{4}$$

$$\text{não existam sub-rotas} \tag{5}$$

Mas somente com as três primeiras restrições (2, 3, 4) é possível que apareçam sub-rotas conforme ilustrado nas Figuras 3 e 4.

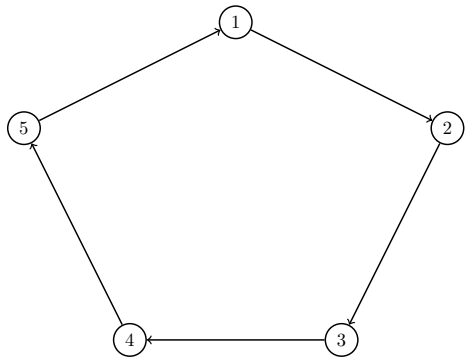

Figura 3: Uma rota

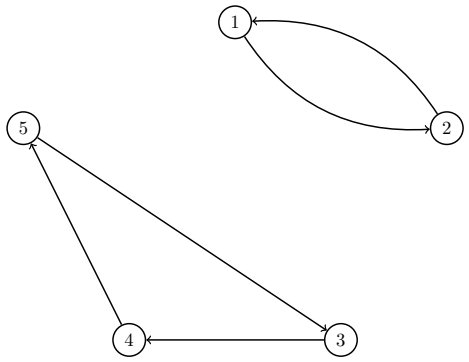

Figura 4: Duas sub-rotas

Para que isso não aconteça é necessário inserir no modelo o que chamamos de restrições de eliminação de sub-rotas. Existem diferentes formulações para evitar o aparecimento de subciclos, para maiores referências pode-se consultar Dantzig et al. ((1954)), Miller et al. ((1960)), Gavish and Graves ((1978)) (monoproduto), Claus ((1984)) (multiproduto) e Finke ((1984)) (dois produtos). A formulação que apresentaremos aqui é baseada no artigo de Gavish and Graves ((1978)) que utiliza a variável de fluxo f_{ij}. Essa variável cria uma quantidade $(n-1)$ de fluxo fictício que deve sair da origem e ser entregue uma unidade em cada nó visitado conforme ilustra a Figura 5, garantindo que o veículo saia da origem, visite os nós definidos para aquela rota e retorna ao ponto inicial, sem formação de subciclos.

Assim, o modelo apresentado é incrementado com as restrições de balanço de fluxo e, considerando os parâmetros e variáveis representados na Tabela 2, podemos reescrevê-lo da seguinte forma:

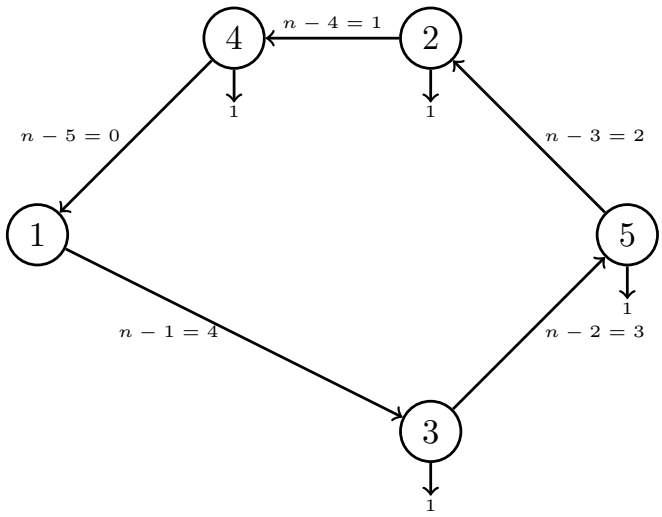

Figura 5: Aplicação da variável de fluxo fictício.

$$\min \sum_{(i,j)\in A} c_{ij}x_{ij} \tag{6}$$

$$\sum_{(i,j)\in A} x_{ij} = 1 \qquad \forall i \in N \setminus \{0\} \tag{7}$$

$$\sum_{(i,j)\in A} x_{ij} = 1 \qquad \forall j \in N \setminus \{0\} \tag{8}$$

$$\sum_{(0,j)\in A} f_{ij} = (n-1) \tag{9}$$

$$\sum_{(i,j)\in A} f_{ij} - \sum_{(j,i)\in A} f_{ji} = 1 \qquad \forall j \in N \setminus \{0\} \tag{10}$$

$$f_{ij} \leq (n-1)x_{ij} \qquad \forall (i,j) \in A \tag{11}$$

$$f_{ij} \geq 0 \qquad \forall (i,j) \in A \tag{12}$$

$$x_{ij} \in \{0,1\} \qquad \forall (i,j) \in A \tag{13}$$

4 Códigos do modelo em MathProg e Python

A seguir são apresentados os códigos dos modelos em MathProg e Python. Além dos modelos, também é apresentado, em Python, o código que pode ser utiliado para gerar instâncias para esse tipo de problema.

Listing 1: Modelo feito em MathProg

```
# numero de nos
param n;

# conjunto de nos

set N := 1..n;
set A := {(i,j) in N cross N: i != j};
param planta default 1;

# posicao geografica dos nos

param posx{N};
param posy{N};
param cur_i;

# distancia Euclidiana entre i e j

param c{(i,j) in A} := sqrt( (posx[i] - posx[j])^2 + (posy[i] - posy[j])^2 );

# x_ij igual a 1 se no j e visitado imediatamente apos no i; 0, caso contrario
var x{A}, binary;

# f_ij qtde de produto ficticio transportada pelo caixeiro viajante no arco (i,j)
var f{A}, >= 0;


# funcao objetvio: minimizacao do comprimento/distancia da rota
minimize of : sum{(i,j) in A} c[i,j] * x[i,j];


# restricoes do grau de cada no: em todo no, so chega um arco e so sai um arco dele
s.t. restricao_arco_saida{i in N}: sum{(i,j) in A} x[i,j] = 1;
s.t. restricao_arco_chegada{j in N}: sum{(i,j) in A} x[i,j] = 1;


# restricoes para eliminacao de subciclos ou subrotas
# o caixeiro sai da planta com n-1 unidades de produto ficticio
# para deixar uma unidade em cada no a ser visitado
s.t. restricao_planta: sum{(i,j) in A: i == planta} f[i,j] = n - 1;


# restricao de balanco de fluxo: o caixeiro chega em um no com uma determinada
# quantidade de produto ficticio, deixa uma unidade deste produto, e sai
# deste no carregando uma unidade a menos.
s.t. restricao_balanco_de_fluxo{j in N: j != planta}: sum{(i,j) in A} f[i,j] = 1 + sum{(j,i) in A} f[j,i];


# restricao de ativacao do arco: o caixeiro so pode usar o arco (i,j)
# para carregar produto ficticio se o arco for contabilizado na funcao objetivo
s.t. restricao_de_ativacao{(i,j) in A}: f[i,j] <= (n-1) * x[i,j];


# otimiza o modelo chamando o resolvedor
solve;
printf '\n\n';
printf 'Custo da rota: %12.2f\n', of;
printf 'Arcos ativos: \n';
for{(i,j) in A: x[i,j] > 0.9}
{
   printf "(%d,%d)\n", i,j;
}
data;
param n := 21;
param : posx posy :=
```

```
  1       40.00        5.00
  2       25.00       46.00
  3       27.00       39.00
  4        3.00       10.00
  5       17.00       20.00
  6       33.00        7.00
  7       32.00        5.00
  8       26.00       22.00
  9       20.00       31.00
 10       31.00       36.00
 11       23.00       45.00
 12       38.00       23.00
 13       14.00       28.00
 14       45.00       21.00
 15        9.00       14.00
 16       19.00       42.00
 17       36.00       29.00
 18        7.00       17.00
 19       46.00        4.00
 20       30.00       26.00
 21       18.00       30.00
;
end;
```

Listing 2: Modelo feito em Python

```
from mip import Model, xsum, minimize, CBC, OptimizationStatus, BINARY
from itertools import product
import matplotlib.pyplot as plt
from math import sqrt
import numpy as np

# numero de clientes,no de identificacao da planta
n = 21
planta = 0


# listas com os indices dos clientes e dos arcos
N = range(n)
A = [(i,j) for (i,j) in product(N,N) if i != j]


# posicao geografica dos nos
posx = np.array([40, 25, 27, 3, 17, 33, 32, 26, 20, 31, 23, 38, 14, 45, 9, 19, 36, 7, 46, 30, 18])
posy = np.array([5, 46, 39, 10, 20, 7, 5, 22, 31, 36, 45, 23, 28, 21, 14, 42, 29, 17, 4, 26, 30])


# distancia Euclidiana entre os nos
c = [ [ sqrt( (posx[i] - posx[j])**2 + (posy[i] - posy[j])**2 ) for j in N ] for i in N]


# declaracao do modelo
model = Model('Problema do caixeiro viajante',solver_name=CBC)


# x_ij igual a 1 se no j e visitado imediatamente apos no i; 0, caso contrario
x = {(i,j) : model.add_var(var_type=BINARY) for (i,j) in A}


# f_ij qtde de produto ficticio transportada pelo caixeiro viajante no arco (i,j)
f = {(i,j) : model.add_var(lb=0.0) for (i,j) in A}


# funcao objetivo: minimizacao do comprimento/distancia da rota
model.objective = minimize(xsum(c[i][j] * x[i,j] for (i,j) in A))


# restricoes do grau de cada no: em todo no, so chega um arco e so sai um arco dele
for i in N:
    model += xsum(x[ii,j] for (ii,j) in A if i == ii) == 1
for j in N:
    model += xsum(x[i,jj] for (i,jj) in A if j == jj) == 1

```

```

# restricoes para eliminacao de subciclos ou subrotas
# o caixeiro sai da planta com n-1 unidades de produto ficticio
# para deixar uma unidade em cada no a ser visitado
#restricoes de ativacao: um cliente i so pode ser atendido por j se j estiver instalado
model += xsum(f[i,j] for (i,j) in A if i == planta) == n - 1


# restricao de balanco de fluxo: o caixeiro chega em um no com uma determinada
# quantidade de produto ficticio, deixa uma unidade deste produto, e sai
# deste no carregando uma unidade a menos.
for j in N:
    if j != planta:
        model += xsum(f[i,jj] for (i,jj) in A if j == jj) == 1 + xsum(f[jj,i] for (jj,i) in A if j == jj)


# restricao de ativacao do arco: o caixeiro so pode usar o arco (i,j)
# para carregar produto ficticio se o arco for contabilizado na funcao objetivo
for (i,j) in A:
    model += f[i,j] <= (n-1) * x[i,j]


# otimiza o modelo chamando o resolvedor
status = model.optimize()
if status == OptimizationStatus.OPTIMAL:
    print("Custo da rota: {:12.2f}".format(sum([c[i][j] * x[i,j].x for (i,j) in A])))
    cur_no = planta
    print("Rota: ")
    while True:
          print("{:3d} ".format(cur_no),end='')
          cur_no = [j for (i,j) in A if i == cur_no and x[i,j].x > 0.9][0]
          if cur_no == planta:
              break;
    print(), print()

    fig, ax = plt.subplots()
    plt.scatter(posx[:],posy[:],marker="o",color='black',s=10,label="nos")
    for i in N:
        plt.text(posx[i],posy[i], "{:d}".format(i))

    for (i, j) in [(i, j) for (i, j) in A if x[i, j].x >= .9]:
        plt.plot((posx[i], posx[j]), (posy[i], posy[j]), linestyle="--", color="black")

    plt.scatter(posx[0],posy[0],marker="^",color='black',s=100,label="planta")
    plt.text(posx[0]+.5,posy[0], "{:s}".format("planta"))
    plt.legend()
    plt.plot()
    plt.savefig("exemplo_solucao.pdf")
    plt.show()
```

5 Novas instâncias

Novas instâncias podem ser geradas a partir do código abaixo:

Listing 3: Código para gerar instâncias

```
from random import seed, randint, sample
import matplotlib.pyplot as plt

import numpy as np
seed(0)

ni,nj = 33,1
I,J = range(ni),range(nj)

coordxi = sample(range(1,50),ni)
coordyi = sample(range(1,50),ni)
coordxj = sample(range(5,45),nj)
coordyj = sample(range(5,45),nj)


```

```
fig, ax = plt.subplots()
plt.scatter(coordxi,coordyi,marker="o",color='black',s=10,label="clientes")
for i in I:
    plt.text(coordxi[i],coordyi[i], "{:d}".format(i + 2))

plt.scatter(coordxj,coordyj,marker="^",color='black',s=100,label="planta")
for j in J:
    plt.text(coordxj[j],coordyj[j], "{:s}".format("planta"))
plt.legend()
plt.plot()
plt.savefig("exemplo_caso_teste.pdf")
#plt.show()


dj = [ [1,coordxj[j],coordyj[j]] for j in J]
di = [ [i+2,coordxi[i],coordyi[i]] for i in I]
dados = np.array(dj + di)

print("param : posx posy :=")
for (i,x,y) in dados:
    print(" {:3d} {:12.2f} {:12.2f}".format(i,x,y))
print(";")
print()
labels = ['posx', 'posy']
for h in range(1,3):
  print("{:s} = np.array([".format(labels[h-1]),end='')
  for i,v in enumerate(dados[:,h]):
      print("{:d}".format(v),end='')
      if i == len(dados) - 1:
         print("])".format(v))
      else:
         print(", ".format(v),end='')
```

Referências

N. Biggs, E. K. Lloyd, and R. J. Wilson. *Graph Theory, 1736-1936*. Oxford University Press, 1986.

A. Claus. A new formulation for the travelling salesman problem. *SIAM Journal on Algebraic Discrete Methods*, 5(1):21–25, 1984.

G. Dantzig, R. Fulkerson, and S. Johnson. Solution of a large-scale traveling-salesman problem. *Journal of the operations research society of America*, 2(4):393–410, 1954.

G. Finke. A two-commodity network flow approach to the traveling salesman problem. *Congresses Numeration*, 41:167–178, 1984.

B. Gavish and S. C. Graves. The travelling salesman problem and related problems. 1978.

R. Matai, S. P. Singh, and M. L. Mittal. Traveling salesman problem: an overview of applications, formulations, and solution approaches. *Traveling salesman problem, theory and applications*, 1 (1):1–25, 2010.

C. E. Miller, A. W. Tucker, and R. A. Zemlin. Integer programming formulation of traveling salesman problems. *Journal of the ACM (JACM)*, 7(4):326–329, 1960.

A. Schrijver. On the history of combinatorioal optimization. *Discrete Optimization*, pages 1–68, 1960.